



Grado en ingeniería informática

Sistemas distribuidos 25/26

Grupo 81

Práctica 1

«Ejercicio evaluable 2»

Chen Xu — 100522395

Miguel Amiel Gil — 100525454

Profesor

Felix García Carballeira

1. Diseño y compilación

1.1. Diseño

1.2. Compilación

Para realizar la compilación del programa se utiliza el comando «make». Para ejecutar el servidor se ha de abrir una terminal separada y correrlo usando «./servidor-sockets 3000». Para la correcta ejecución de los clientes asume el uso del puerto 3000 por el servidor. Para correr los clientes hemos incluido un archivo .sh llamado «run.sh» para correr los tests del cliente no distribuido y «run_d.sh» para correr los distintos clientes de la versión distribuida.

```
// Terminal 1
make
./servidor-sock 3000 # Hacer después de la línea uno de terminal 2
// Terminal 2
./run.sh # Ejecutamos los tests que comprueban la funcionalidad de las
instrucciones de claves.c
./run_d.sh # Ejecutamos los tests de la parte distribuida
```

2. Batería de pruebas

2.1. Parte no distribuida

ID	Test	Explicación	Entrada	Salida
1	Añadir clave (set_value)	Inserción correcta de una nueva clave inexistente.	key: «test1» value1: «test1_v1» N_v2: 4 V_v2: {1.5, ...}	0
2	Clave repetida	Error al intentar insertar una clave que ya existe en el sistema.	key: «test1» value1: «test2_v1»	-1
3	N_value2 fuera de rango	Error de validación cuando el tamaño del array excede el límite (33).	N_value2: 33	-1
4	Recuperar clave (get_value)	Lectura exitosa de datos previamente insertados.	Strings/Arrays vacíos (Buffer de salida)	0
5	Modificar inexistente	Error al intentar modificar una clave que no está en la base de datos.	Strings/Arrays vacíos	-1
6	Modificar existente	Actualización correcta de los valores para una clave ya presente.	key: «test1» V_value2: {2, 3, 4, 5, 6}	0
7	Modificar inexistente (modify_value)	Error al intentar modificar una clave que no existe.	key: «no_existo» value1: «test7_v1» N_v2: 4 V_v2: {1.5, ...}	0
8	N_value2 fuera de rango (modify_value)	Error de validación en modificación por tamaño de array excesivo.	key: «test1» N_v2: 33	-1
9	Comprobar existencia	Verificación de que una clave existe en el sistema.	key: «test1»	1
10	Comprobar inexistencia	Verificación de que una clave no existe en el sistema.	key: «no_existo»	0
11	Borrar clave (delete_key)	Eliminación correcta de una clave existente.	key: «test1»	0
12	Borrar clave inexistente	Error al intentar borrar una clave que no existe o ya fue borrada.	key: «test1»	-1
13	Borrar todo (destroy_value)	Eliminación de todas las tuplas almacenadas.	No hay argumentos	0

2.2. Parte distribuida

En este caso, para probar el correcto funcionamiento de la concurrencia y de las llamadas a la API, hemos elaborado varios ficheros app-cliente.c que realizan peticiones conflictivas, por ejemplo, set_value y destroy. De esta forma, podemos observar cómo

dependiendo del orden en el que se procesan las peticiones por la pool de threads, el resultado obtenido es uno u otro.

Estos tests se utilizaron para comprobar la concurrencia. Sin embargo, por el hecho de ser un servidor concurrente, la salida no está fijada a ser siempre la misma. Dependiendo del orden de ejecución que siguió la thread pool, la salida puede ser una u otra.

Test ID	Explicación del test	Iteración N	Iteración N + K (K > 0)
1	Ejecutar archivo app-cliente3.c sin haber abierto el servidor.	No se pudo abrir la cola del servidor: No such file or directory No se ha podido enviar la petición: Bad file descriptor	No necesitamos otra iteración ya que no estamos evaluando la concurrencia, sino la generación de los códigos de error esperados.
2	Fichero app-cliente-2.c. Copia exacta del fichero app-cliente.c que se utiliza para comprobar el funcionamiento de las llamadas a la API por un único thread. Las mismas que en los tests definidos para la parte no distribuida.	Las mismas salidas que en la parte no distribuida. De esta forma, se puede comprobar que la API está bien elaborada. Procedemos a analizar la concurrencia.	No necesitamos otra iteración, estamos comprobando si funcionan las llamadas desde la API por un único thread.
3	Ejecución simultánea de app-cliente-3.c (borrado «test1») y app-cliente-4.c (registro «test1»). Dependiendo del orden, la key se registra y borra, o se intenta borrar primero.	Iteración 1 cliente3: Se borró la key cliente4: El resultado del test1 fue 0 /RESULTADO ESPERADO 0	Iteración 3 cliente3: No existe key cliente4: El resultado del test1 fue 0 /RESULTADO ESPERADO 0
4	Ejecución simultánea de app-cliente-3.c (borrado), app-cliente-4.c (registro) y app-cliente-5.c (recupera datos).	Iteración 1 cliente3: No existe key cliente4: Resultado 0 cliente5: Se recuperaron los resultados!: value1 test1_value1, N_value2 4, paquete{1, 2, 3}	Iteración 2 cliente3: Se borró la key cliente4: Resultado 0 cliente5: No se pudo recuperar la info
5	Ejecución simultánea de app-cliente-3.c, app-cliente-4.c y app-cliente-6.c. El cliente 6 intenta modificar «test1». Se espera que la clave se borre, cree o modifique según el orden.	Iteración 1 cliente3: No existe key cliente4: El resultado del test1 fue 0 /RESULTADO ESPERADO 0 cliente6: No se pudo modificar la clave la info	Iteración 4 cliente3: No existe key cliente4: El resultado del test1 fue 0 /RESULTADO ESPERADO 0 cliente6: Se modificaron la clave!